

GUIA MASTER DE ARQUITETURA THREE.JS & ECOSSISTEMA

Análise Abrangente de Bibliotecas, Movimento/Foot IK, Neve/Areia, NavMesh 3D & Ponto Exato de Transição para o React

Resumo Executivo & Escopo do Documento:

Este documento compila a análise técnica aprofundada de todas as bibliotecas do ecossistema **Three.js (WebGL2 e WebGPU/TSL)** para o seu jogo de física, magia elemental e exploração. Ele detalha as vantagens de cada biblioteca, apresenta a solução para os 4 gargalos críticos (Foot IK no terreno, NavMesh 3D de NPCs, consultas rápidas BVH e deformação visual de neve/areia) e estabelece o **momento exato de transição do código para o React (React Three Fiber)**.

1. Panorama dos 6 Frameworks Baseados em Three.js

A escolha da arquitetura web determina o custo de produção, a velocidade de desenvolvimento e a facilidade de integrar física, UI, XR e editores visuais:

Framework	Abordagem	Vantagens Principais	Limitações	Veredito Analítico
React Three Fiber (R3F)	Custom Renderer (React)	Ecossistema gigante (pmndrs), Drei, Rapier, UIKit 3D, Post-processing avançado, milhares de componentes prontos.	Requer decisões de arquitetura e conhecimento do ecossistema React.	■ Escolha Principal para jogos web modernos e complexos.
A-Frame	ECS Declarativo (HTML)	Porta de entrada mais simples para WebXR (VR/AR), inspetor visual embutido (Ctrl+Alt+I), semântica limpa.	Ergonomia menor para lógica de jogo sistêmica pesada e apps frontend grandes.	Ideal para WebXR , protótipos rápidos e iniciantes.
Needle Engine	Editor-First (Unity/Blender)	Pipeline de estúdio com Unity/Blender, multiplayer & VOIP nativos, otimização automática de assets, WebXR.	Licenciamento comercial (exige licença Pro para produtos comerciais).	Excelente para estúdios e equipes com artistas 3D.
Threlte	Declarativo (Svelte)	Reatividade de alta performance do Svelte, módulos de Rapier, XR, Theatre.js e gamepad frame-accurate.	Comunidade e volume de exemplos menores que o R3F.	Melhor opção para Svelte.
TresJS	Custom Renderer (Vue 3)	Sintaxe nativa do Vue 3, módulos Cientos e Rapier, integração com Nuxt DevTools e suporte rápido ao Three.js.	Ecossistema menor que o pmndrs.	Melhor opção para Vue/Nuxt.
enable3d	Toolkit (Phaser 3 + Three)	Ponte direta entre a lógica 2D do Phaser 3 e a renderização 3D do Three.js + Ammo.js.	Defasagem de versão do Three.js (0.171), dependência em Ammo.js antigo.	Restrito a jogos 2.5D arcade em Phaser.

2. Em Que Momento Transicionar para o React (E Por Quê?)

Seu código atual em teste laboratório é escrito em **JavaScript Vanilla puro**. Embora o JS puro seja excelente para validar a física inicial e testes unitários, ele encontra um gargalo de desenvolvimento conforme o jogo se expande. Veja o roteiro de transição:

ROTEIRO DA TRANSIÇÃO PARA O REACT & REACT THREE FIBER (R3F):

Fase 1 (Hoje / JS Vanilla): Mantenha em JS Vanilla enquanto desenvolve os protótipos de física com Rapier, raycast com three-mesh-bvh e o controlador de personagem three-player-controller.

PONTO DE VIRADA (Quando transicionar para o React): Você DEVE transicionar para o React + R3F no momento em que iniciar as seguintes áreas:

1. **Árvore de Diálogos & HUD Complexo:** Gerenciar as caixas de diálogo do Ink.js, opções de escolha, retrato de NPCs e inventário via manipulação manual do DOM (JS Vanilla) torna o código prolixo e suscetível a bugs. No React, isso é resolvido com componentes e Zustand.
2. **Controlador Avançado de Personagem (ecctrl):** Se desejar gravidade personalizada (andar nas paredes, atração por objetos, zonas de levitação giratórias), a biblioteca ecctrl combina R3F + Rapier nativamente.
3. **Cutscenes & Animação (Theatre.js):** Integrar a timeline visual do Theatre.js para animar propriedades de luzes, câmeras e materiais é infinitamente mais produtivo com bindings reativos de React.
4. **Gerenciamento de Memória Automático:** No JS Vanilla, remover objetos do jogo exige chamar dispose() manualmente em geometrias, materiais e texturas para evitar vazamentos de memória. No R3F, a desalocação é automática ao desmontar o componente.

3. Detalhamento do Stack Recomendado & Suas Vantagens

A. Aceleração Espacial: three-mesh-bvh (Adicionar Imediatamente)

- **Vantagens:** Acelera o Raycaster padrão do Three.js em até 100x usando árvores Bounding Volume Hierarchy.
- **Aplicação:** Usado para detectar o raio sob cada pé (Foot IK), identificar o material do solo (areia/neve), colisão suave de câmera e seleção rápida de objetos levitáveis.

B. Movimentação & Ajuste do Terreno: three-player-controller + Foot IK

- **Vantagens:** Resolve o problema de pés flutuando ou afundando no terreno. Ajusta automaticamente a sola do pé, joelhos e quadril em dunas, neve irregular e escadas.
- **Aplicação:** Permite acionar o carimbo da pegada exatamente no momento em que a sola toca a superfície.

C. Navegação 3D de NPCs: recast-navigation (`@recast-navigation/three`)

- **Vantagens:** Port em WebAssembly do Recast/Detour (padrão AAA). Suporta NavMesh tridimensional em múltiplos andares, rampas, escadas e obstáculos temporários (TileCache).
- **Aplicação:** Acompanhante e inimigos perseguindo/fugindo pelo laboratório sem travar em obstáculos dinâmicos movidos por levitação.

D. Divisão de VFX: three.quarks vs @newkrok/three-particles

- **three.quarks (VFX Autoral):** Fogo nas mãos, orbes mágicos, auras, explosões e magias próximas. Possui editor visual Unity Shuriken.
- **@newkrok/three-particles (Clima em Massa via WebGPU/TSL):** Nevascas, tempestades de areia, fuligem e poeira contínua rodando diretamente na GPU sem sobrecarregar a CPU.

E. Turbulência de Vento: three-fluid-fx

- **Vantagens:** Solucionador de fluidos 2D/3D para campos de velocidade e densidade.
- **Aplicação:** Vórtices de vento saindo das mãos, ondas de choque de levitação e distorção de calor do fogo.

4. Sistema Próprio de Deformação de Neve, Areia e Lama

Não existe uma biblioteca pronta única no mercado que resolva rastros de neve e areia com deformação e física. A arquitetura correta une 4 módulos do stack:

FLUXO DO SURFACE INTERACTION SYSTEM:

- Evento de Contato do Pé:** O controlador de animação dispara o evento no exato instante do pé plantado.
- Raycast BVH:** O three-mesh-bvh identifica as coordenadas exatas, a normal e o tipo do solo (neve, areia, lama).
- Foot IK:** Ajusta a inclinação da sola e dos joelhos do personagem.
- Carimbo no RenderTarget (GPU):** Grava em um RenderTarget móvel a máscara da sola (Canal R: profundidade, Canal G: bordas acumuladas, Canal B: umidade/compactação).
- Shader do Terreno:** Desloca os vértices do solo para baixo, altera o normal map e reduz o roughness na neve compactada.
- Partículas & Som:** O three.quarks emite um pequenos "puff" de neve/areia subindo, enquanto o AudioSystem toca o som do passo.

5. Matriz Consolidada de Bibliotecas & Comandos de Instalação

Categoria	Biblioteca Recomendada	Função Específica no Jogo	Fase Recomendada
Física Geral	@dimforge/rapier3d-compatible	Corpos rígidos, levitação, impulsos	Fase 1 (Atual)
Aceleração Espacial	three-mesh-bvh	Raycasts ultrarrápidos, solo, câmera	Fase 1 (Atual)
Movimento/Foot IK	three-player-controller	Personagem 3D e pés adaptativos no terreno	Fase 1 (Atual)
Navegação NPCs	recast-navigation	NavMesh 3D com escadas e TileCache	Fase 2 (Médio Prazo)
VFX Principal	three.quarks	Fogo, auras, explosões, poderes	Fase 1 (Atual)
VFX Clima em Massa	@newkrok/three-particles	Nevascas, tempestades de areia em GPU	Fase 2 (Médio Prazo)
Vento/Fluidos	three-fluid-fx	Campos de velocidade de vento e vórtices	Fase 2 (Médio Prazo)
Arquitetura ECS	koota	Gestão de projéteis e entidades em massa	Fase 3 (Transição React)
Diálogos	inkjs / Yarn Spinner	História ramificada e decisões	Fase 3 (Transição React)
Cutscenes	@theatre/core	Timeline visual de câmeras e cenários	Fase 3 (Transição React)

Comando Único de Instalação (para Fase 1 e 2):

